

txt2tex Reference

Write math like you're at a whiteboard. Get L^AT_EX you can hand in.

<https://github.com/jmf-pobox/txt2tex>

Document Structure

=== Title ===	<code>\section *{Title}</code>
* Solution N **	Bold heading with spacing
(a) Text	Part label
TEXT: paragraph	Prose block (smart formula detection)
LATEX: <code>\cmd</code>	Raw L ^A T _E X passthrough
PAGEBREAK:	Page break
LINEBREAK:	Vertical gap (<code>\medskip</code>) — between blocks, no page break
TITLE: ...	Document title
AUTHOR: ...	Author name
DATE: ...	Date string
BIBLIOGRAPHY: f.bib	BibTeX file
[cite key]	<code>\citep {key}</code>

Identifier Decoration

<code>count'</code>	<i>count'</i> (after-state)
<code>in?</code>	<i>in?</i> (input)
<code>out!</code>	<i>out!</i> (output)
<code>x?'</code>	<i>x?'</i> (runs may combine)
<code>'sunk'</code>	'sunk' (string literal)

Decoration attaches to any identifier. Reserved words (`theta`, `defs`, `hide`, `project`, `Delta`, `Xi`) cannot be decorated.

Logic

<code>p land q</code>	$p \wedge q$
<code>p lor q</code>	$p \vee q$
<code>lnot p</code>	$\neg p$
<code>p => q</code>	$p \Rightarrow q$
<code>p <=> q</code>	$p \Leftrightarrow q$
<code>forall x : S P</code>	$\forall x : S \bullet P$
<code>exists x : S P</code>	$\exists x : S \bullet P$
<code>exists_1 x : S P</code>	$\exists_1 x : S \bullet P$
<code>lambda x : S E</code>	$\lambda x : S \bullet E$
<code>mu x : S P</code>	$\mu x : S \bullet P$

Note: Use `land`, `lor`, `lnot`. English `and/or/not` are not supported.

Sets & Types

<code>x elem S</code>	$x \in S$
<code>x notin S</code>	$x \notin S$
<code>A subset B</code>	$A \subseteq B$
<code>A union B</code>	$A \cup B$
<code>A inter B</code>	$A \cap B$
<code>A setminus B</code>	$A \setminus B$
<code>power S</code>	$\mathcal{P} S$
<code>F S</code>	$\mathcal{F} S$
<code>{x : S P}</code>	Set comprehension
<code>A cross B</code>	$A \times B$
<code>N / Z / N1</code>	$\mathbb{N} / \mathbb{Z} / \mathbb{N}_1$
<code>n..m</code>	$n \dots m$

Relations

<code>A <-> B</code>	$A \leftrightarrow B$
<code>x -> y</code>	$(x \mapsto y)$
<code>dom R / ran R</code>	$\text{dom } R / \text{ran } R$
<code>A dres R</code>	$A \triangleleft R$
<code>A dsub R</code>	$A \triangleleft R$
<code>R rres B</code>	$R \triangleright B$
<code>R rsub B</code>	$R \triangleright B$
<code>R oplus S</code>	$R \oplus S$ (override)
<code>R o9 S</code>	$R \circ_9 S$ (forward comp.)
<code>R comp S</code>	$R \circ_9 S$ (backward comp.)

Functions

<code>A pfun B</code>	$A \mapsto B$
<code>A fun B</code>	$A \longrightarrow B$ (total)
<code>A pinj B</code>	$A \mapsto B$
<code>A inj B</code>	$A \mapsto B$
<code>A surj B</code>	$A \twoheadrightarrow B$
<code>A bij B</code>	$A \mapsto B$
<code>A ffun B</code>	$A \mapsto B$ (finite)
<code>f(x)</code>	Function application

Sequences

<code><> / <a, b></code>	$\langle \rangle / \langle a, b \rangle$
<code>s ^ t</code>	$s \frown t$ (concatenation)
<code>seq S / seq1 S</code>	$\text{seq } S / \text{seq}_1 S$
<code># s</code>	$\#s$ (length)

Z Notation Blocks

<code>given A, B</code>	Given types: $[A, B]$
<code>T ::= a b</code>	Free type
<code>name == expr</code>	Abbreviation

```
axdef / schema Name / gendef [X]
  declarations  Variable declarations
where          Separator
  predicates    Constraining predicates
end            Close the block
```

Schema Inclusion & Calculus

```
Delta Counter  ΔCounter (before/after)
Xi Card        ∃Card (read-only)
SName          bare inclusion
theta S        θS (state binding)
theta S'       θS' (after-state)
```

```
Name defs RHS — horizontal definition:
Name defs Schema      Name ≐ Schema
N defs Delta C        N ≐ ΔC
N defs [d : T | P]    inline schema text
N[X] defs G[X]        generic form
```

```
defs RHS operators (schema calculus):
S[old/new]          rename component
S[a/b, c/d]         multiple renames
S ; T               S ∖ T (composition)
S » T               S >> T (piping)
S hide (x, y)       S \ (x, y)
S project T         S | T
```

Proof Tree Syntax

```
indent          Premises indented under conclusion
[rule]          Justification label
[rule [n]]      Discharge assumption n
[n] expr        Boxed assumption labelled n
::              Sibling premises
```

```
Rules: land intro, land elim 1/2, lor intro 1/2, lor elim,
=> intro/elim, <=> intro/elim, lnot intro/elim, forall
intro/elim, exists intro/elim.
```

Usage

```
txt2tex f.txt          → f.tex + f.pdf
txt2tex f.txt --tex-only → f.tex only
txt2tex -i             Interactive REPL
txt2tex --check-env     Verify dependencies
```

txt2tex Proofs — By Example

Each example shows what you type and what txt2tex renders.

How Proof Trees Work

In txt2tex, proof trees are written **bottom-up**: the conclusion is the first line, and its premises are indented below it. Think of it as reading the inference rule from conclusion to justification:

conclusion	First line (least indented)
premise	Indented = supports the line above
[rule]	Justification in brackets
[n] expr	Boxed assumption (labelled n)
[rule [n]]	Discharge assumption n
::	Marks sibling premises

Deeper indentation = deeper in the tree. Two premises at the same indent level are siblings that jointly support their parent.

Truth Table Prop. Logic

You write:

```
TRUTH TABLE:
p | q | p => q
T | T | T
T | T | T
T | F | F
F | T | T
F | F | T
```

You get:

p	q	$p \Rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Equivalence Chain Prop. Logic

You write:

```
EQUIV:
lnot (p land q)
lnot p lor lnot q [De Morgan]
```

You get:

$$\neg (p \wedge q) \\ \Leftrightarrow \neg p \vee \neg q \quad [\text{De Morgan}]$$

EQUIV: joins steps with $\Leftrightarrow$. Use for *predicate* equivalences.

Modus Ponens Proof Trees

You write:

```
PROOF:
  q [=> elim]
  p => q
  p
```

You get:

$$\frac{p \Rightarrow q \quad p}{q} \Rightarrow \text{-elim}$$

$\wedge$ -intro Proof Trees

You write:

```
PROOF:
  p land q [land intro]
  p
  q
```

You get:

$$\frac{p \quad q}{p \wedge q} \wedge \text{-intro}$$

$\Rightarrow$ -intro with discharge Proof Trees

You write:

```
PROOF:
  p => p [=> intro [1]]
  [1] p
```

You get:

$$\frac{\lceil p \rceil^{[1]}}{p \Rightarrow p} \Rightarrow \text{-intro}^{[1]}$$

[1] p = boxed assumption. [=> intro [1]] discharges it.

$\vee$ -elim / case analysis Proof Trees

You write:

```
PROOF:
  r [lor elim]
  p lor q
  :: p => r
    [1] p
    r [from p]
  :: q => r
    [2] q
    r [from q]
```

You get:

$$\frac{p \vee q \quad \frac{\lceil p \rceil^{[1]}}{r} \quad \frac{\lceil q \rceil^{[2]}}{r}}{r} \vee \text{-elim}$$

:: = sibling premises (case arms).

Equality Chain Induction

You write:

```
EQUAL:
0 + 0
0 [0+0 = 0]
length(<>)
[length(<>) = 0]
```

You get:

$$0 + 0 \\ = 0 \quad [0+0 = 0] \\ = \text{length}(\langle \rangle) \quad [\text{length}(\langle \rangle) = 0]$$

EQUAL: = expression chains (=). Use for N/sets/sequences.

txt2tex Relational Databases — By Example

DAT course notation: primary keys, relational algebra, bindings, GROUP/UNGROUP.

Primary Keys

Mark a primary-key attribute with **pk** in a named schema body. The PK annotation sits *outside* the Z box so fuzz can type-check the schema cleanly.

You write:

```
schema Class
  pk class : ClassName
  country : CountryName
  bore    : N
end
```

You get:

```
Class
class : ClassName
country : CountryName
bore : N
```

$\text{PK}(\text{Class}) = \{class\}$

Composite: two **pk** lines $\Rightarrow \text{PK}(S) = \{a, b\}$. Comma-separated names on one **pk** line also work. **pk** is rejected in **axdef** / **gendef** / inline schema text.

Relational Algebra

$\text{sigma}[p](R)$	$\text{Restrict}_p(R)$ — restrict
$\text{pi}[A, B](R)$	$\text{Project}\{A, B\}(R)$ — project
$\text{rho}[A \text{ as } B](R)$	$\text{Rename}_{A \rightarrow B}(R)$ — rename
$R \text{ bowtie } S$	$R \otimes S$ — natural join
$R \text{ bowtie } [p] S$	$\text{Join}_p(R, S)$ — theta-join
$R \text{ div } S$	$R \text{ div } S$ — division

sigma/pi/rho bind at atom level (prefix + args). **bowtie** and **div** are infix at cross-product precedence.

Naming a query. Use **Name == expr** (Z abbreviation) for both pure-Z and DAT-bearing right-hand sides. **txt2tex** inspects

the RHS: when it contains a DAT construct (algebra, binding, GROUP/UNGROUP), the abbreviation emits as `\noindent$...$` outside any Z block; otherwise it emits inside a **zed** block.

Fuzz note: algebra expressions emit *outside* any Z environment, so fuzz silently skips them — your schemas and axdefs in the same document still type-check cleanly. Do *not* wrap algebra inside **zed/axdef/schema** blocks; fuzz parses block contents strictly as Z and will reject the algebra subscripts. **Multi-line:** infix algebra operators wrap across lines (bare newline or `\`). Break *before* **setminus** — `\` is its token.

You write:

```
BigGuns == pi[class, country](
  sigma[bore >= 16](Class))
```

You get:

$\text{BigGuns} \hat{=}$ $\text{Project}\{class, country\}(\text{Restrict}_{bore \geq 16}(\text{Class}))$

Z Bindings & Set Comprehension

Z RM §3.7 binding brackets construct labelled tuples.

<code>{ name == e }</code>	$\langle name == e \rangle$
<code>{ a == e, b == f }</code>	multiple components
<code>{ }</code>	empty binding

Multi-typed comprehension with binding:

You write:

```
{ s : Ship; c : Class |
  s.class = c.class .
  { | name == s.name | } }
```

You get:

$\{s : \text{Ship}; c : \text{Class} \mid s.class = c.class \bullet \langle name == s.name \rangle\}$

Use `;` to separate variable-type pairs inside `{ }`. Binding components are **comma-separated** (Z RM §3.7).

Fuzz note: bindings emit *outside* any Z environment, so fuzz silently skips them. Schemas and axdefs in the same document still

type-check. Do *not* place a binding inside a **zed/axdef/schema** block; fuzz parses block contents strictly and rejects `==` inside `\{...\}` even though both the brackets and the operator are defined in **fuzz.sty**.

GROUP & UNGROUP

Date's nested-relation operators.

R group (<code>{A,B}</code> as alias)	$R \text{ GROUP}(\{A, B\} \text{ AS } alias)$
R ungroup alias	$R \text{ UNGROUP } alias$

You write:

```
Members group ({name,email} as contact)
Members ungroup contact
```

You get:

$Members \text{ GROUP } (\{name, email\} \text{ AS } contact)$
 $Members \text{ UNGROUP } contact$

Fuzz note: GROUP/UNGROUP emit *outside* any Z environment, so fuzz silently skips them. Do *not* place these inside **zed/axdef/schema** blocks — the `\mathop` wrapping and Date braces sit outside Z's grammar and fuzz rejects them in block contents.

group/ungroup use `\mathop{\mathrm{...}}` — kernel LaTeX, no extra package.

DAT Quick Reference

pk a : T	primary key in schema
pk a, b : T	composite pk
sigma[p](R)	restrict
pi[A](R)	project
rho[A as B](R)	rename attr
R bowtie S	natural join ($\otimes$)
R div S	division
{ a == e }	binding bracket
{S; C P . E}	multi-typed set comp
R group ({A} as x)	group attr
R ungroup x	ungroup attr